

LH Nautical – Questao 2.1

Codigo Python: geracao automatica de schema PostgreSQL a partir dos CSVs

Analise de Dados

Agosto de 2026

Questao 2 - Schema (Engenharia de Dados)

Objetivo: script em Python 3 puro (apenas biblioteca padrao: csv, os, re, datetime, argparse) que le todos os CSVs de um diretorio, infere o tipo de cada coluna e gera um unico arquivo schema.sql com os comandos CREATE TABLE compatíveis com PostgreSQL.

Logica de inferencia de tipo (por coluna):

1. Percorre todos os valores da coluna no arquivo inteiro (nao usa amostra parcial, para nao errar tipo por causa de valores raros no fim do arquivo).
2. Classifica na seguinte ordem de prioridade: BOOLEAN -> TIMESTAMP -> DATE -> INTEGER/BIGINT -> NUMERIC(18,4) -> VARCHAR/TEXT.
3. Detecta se a coluna aceita nulo (valores vazios no CSV) e adiciona NOT NULL quando nao houver nenhum valor vazio.
4. Colunas chamadas id viram PRIMARY KEY automaticamente.
5. Campos de texto curtos (<= 255 caracteres) viram VARCHAR(n) com folga; textos mais longos viram TEXT.
6. O nome da tabela e derivado do nome do arquivo CSV (sem extensao).

Resultado da execucao real (contra os 24 CSVs do desafio): 24 tabelas geradas com sucesso, com deteccao correta de tipos (ex.: tax_id virou BIGINT por exceder o limite de INTEGER; expected_delivery_at virou DATE enquanto placed_at virou TIMESTAMP; colunas com valores vazios como complement e trade_name ficaram sem NOT NULL).

Codigo-fonte (generate_schema.py)

```
#!/usr/bin/env python3
"""
generate_schema.py
-----
LH Nautical - Geracao automatica de schema PostgreSQL a partir dos CSVs.

Objetivo:
    Ler todos os arquivos .csv de um diretorio, inferir o tipo de dado
    de cada coluna (INTEGER, BIGINT, NUMERIC, BOOLEAN, DATE, TIMESTAMP
    ou TEXT/VARCHAR) e gerar um unico arquivo "schema.sql" com os
    comandos CREATE TABLE correspondentes, prontos para rodar num
    banco PostgreSQL.

Restricoes seguidas (premissas obrigatorias do desafio):
    - Somente Python 3 puro / biblioteca padrao (csv, os, re, datetime).
    - Nao usa pandas, dask, polars ou qualquer lib externa.
    - Banco de destino: PostgreSQL (tipos e sintaxe compatíveis).
```

Como usar:

```
python3 generate_schema.py --input-dir lh_nautical_csv --output schema.sql
```

Autor: Pipeline de Dados - LH Nautical

```
"""

import csv
import os
import re
import argparse
from datetime import datetime

# -----
# Configuracoes de inferencia de tipo
# -----

# Quantas linhas amostrar por tabela para inferir o tipo das colunas.
# Usar None para ler o arquivo inteiro (mais preciso, porem mais lento
# em arquivos muito grandes). Para este desafio, lemos o arquivo
# inteiro para garantir precisao no tamanho de VARCHAR e na deteccao
# de nulos.
SAMPLE_SIZE = None

# Padroes de data/hora aceitos (formato ISO, como vem no ERP)
DATE_FORMAT = "%Y-%m-%d"
TIMESTAMP_FORMATS = (
    "%Y-%m-%d %H:%M:%S",
    "%Y-%m-%dT%H:%M:%S",
)

BOOLEAN_TRUE = {"true", "t", "1", "yes"}
BOOLEAN_FALSE = {"false", "f", "0", "no"}

# Nomes de colunas que, por convencao do ERP, sao chave primaria
# (quando a coluna se chama exatamente "id")
PRIMARY_KEY_COLUMN = "id"

class ColumnStats:
    """Acumula as estatisticas necessarias para inferir o tipo de uma coluna."""

    def __init__(self, name):
        self.name = name
        self.has_null = False
        self.max_length = 0
        self.all_int = True
        self.all_float = True
        self.all_bool = True
        self.all_date = True
        self.all_timestamp = True
        self.max_abs_int = 0
        self.seen_any_value = False

    def observe(self, raw_value):
```

```

"""Atualiza as estatísticas da coluna com um novo valor bruto (string)."""
value = raw_value.strip() if raw_value is not None else ""

if value == "":
    self.has_null = True
    return # valores vazios não contam para inferência de tipo

self.seen_any_value = True
self.max_length = max(self.max_length, len(value))

# --- inteiro ---
if self.all_int:
    if re.fullmatch(r"-?\d+", value):
        self.max_abs_int = max(self.max_abs_int, abs(int(value)))
    else:
        self.all_int = False

# --- float / numeric ---
if self.all_float:
    try:
        float(value)
    except ValueError:
        self.all_float = False

# --- boolean ---
if self.all_bool:
    if value.lower() not in BOOLEAN_TRUE and value.lower() not in BOOLEAN_FALSE:
        self.all_bool = False

# --- date ---
if self.all_date:
    try:
        datetime.strptime(value, DATE_FORMAT)
    except ValueError:
        self.all_date = False

# --- timestamp ---
if self.all_timestamp:
    parsed_ok = False
    for fmt in TIMESTAMP_FORMATS:
        try:
            datetime.strptime(value, fmt)
            parsed_ok = True
            break
        except ValueError:
            continue
    if not parsed_ok:
        self.all_timestamp = False

def infer_sql_type(self):
    """Decide o tipo SQL (PostgreSQL) mais adequado, na ordem de prioridade:
    boolean > timestamp > date > integer > numeric > texto.
    A ordem evita que, por exemplo, "0"/"1" vire integer em vez de boolean,
    e que datas (que também "parecem" texto) sejam corretamente detectadas
    antes de cair no fallback texto.
    """

```

```

if not self.seen_any_value:
    # Coluna 100% vazia no arquivo inteiro -> assume texto generico
    return "TEXT"

if self.all_bool:
    return "BOOLEAN"
if self.all_timestamp:
    return "TIMESTAMP"
if self.all_date:
    return "DATE"
if self.all_int:
    # BIGINT se o valor exceder o limite do INTEGER padrao do Postgres
    if self.max_abs_int > 2_147_483_647:
        return "BIGINT"
    return "INTEGER"
if self.all_float:
    return "NUMERIC(18,4)"

# Fallback: texto. Usa VARCHAR(n) com folga quando o campo e curto
# (tipicamente codigos/nomes), senao usa TEXT para nao limitar
# campos livres/longos (ex.: descricoes, notas, URIs).
if self.max_length <= 255:
    return f"VARCHAR({max(self.max_length, 1) * 2})"
return "TEXT"

def sniff_delimiter(sample_text):
    """Detecta automaticamente o delimitador do CSV (';' '\t')."""
    try:
        dialect = csv.Sniffer().sniff(sample_text, delimiters=";\t")
        return dialect.delimiter
    except csv.Error:
        return "," # fallback seguro

def analyze_csv(filepath):
    """Le um CSV e retorna (lista_de_colunas, dict[nome_coluna] = ColumnStats)."""
    with open(filepath, "r", encoding="utf-8", newline="") as f:
        sample = f.read(4096)
        f.seek(0)
        delimiter = sniff_delimiter(sample)

        reader = csv.reader(f, delimiter=delimiter)
        header = next(reader)
        header = [h.strip() for h in header]

        stats = {col: ColumnStats(col) for col in header}

        row_count = 0
        for row in reader:
            # Protege contra linhas com numero de colunas diferente do header
            if len(row) < len(header):
                row = row + [""] * (len(header) - len(row))
            elif len(row) > len(header):
                row = row[: len(header)]

```

```

        for col_name, raw_value in zip(header, row):
            stats[col_name].observe(raw_value)

        row_count += 1
        if SAMPLE_SIZE is not None and row_count >= SAMPLE_SIZE:
            break

    return header, stats, row_count

def sanitize_table_name(filename):
    """Deriva o nome da tabela a partir do nome do arquivo CSV."""
    name = os.path.splitext(os.path.basename(filename))[0]
    name = re.sub(r"[^a-zA-Z0-9_]", "_", name).lower()
    return name

def build_create_table_sql(table_name, header, stats):
    """Monta o comando CREATE TABLE para uma tabela, coluna a coluna."""
    lines = [f'CREATE TABLE IF NOT EXISTS "{table_name}" (']

    col_definitions = []
    for col_name in header:
        col_stat = stats[col_name]
        sql_type = col_stat.infer_sql_type()
        nullable_sql = "" if col_stat.has_null else " NOT NULL"

        # Regra simples de chave primaria: coluna chamada "id"
        if col_name == PRIMARY_KEY_COLUMN:
            col_definitions.append(f'        "{col_name}" {sql_type} PRIMARY KEY')
        else:
            col_definitions.append(f'        "{col_name}" {sql_type}{nullable_sql}')

    lines.append(",\n".join(col_definitions))
    lines.append(");")
    return "\n".join(lines)

def generate_schema(input_dir, output_file):
    """Percorre todos os CSVs do diretorio e escreve o schema.sql final."""
    csv_files = sorted(f for f in os.listdir(input_dir) if f.lower().endswith(".csv"))

    if not csv_files:
        raise SystemExit(f"Nenhum arquivo .csv encontrado em: {input_dir}")

    output_blocks = [
        "-- =====",
        "-- LH Nautical - Schema PostgreSQL gerado automaticamente",
        f"-- Gerado em: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        f"-- Total de tabelas: {len(csv_files)}",
        "-- Script: generate_schema.py (Python 3 puro, sem libs externas)",
        "-- =====",
        "",
    ]

    summary = []

```

```

for csv_file in csv_files:
    filepath = os.path.join(input_dir, csv_file)
    table_name = sanitize_table_name(csv_file)

    header, stats, row_count = analyze_csv(filepath)
    create_sql = build_create_table_sql(table_name, header, stats)

    output_blocks.append(f"-- Fonte: {csv_file} ({row_count} linhas, {len(header)} colunas")
    output_blocks.append(create_sql)
    output_blocks.append("")

    summary.append((table_name, len(header), row_count))

with open(output_file, "w", encoding="utf-8") as f:
    f.write("\n".join(output_blocks))

return summary

def main():
    parser = argparse.ArgumentParser(
        description="Gera um schema.sql (PostgreSQL) a partir dos CSVs de um diretorio."
    )
    parser.add_argument(
        "--input-dir", default="lh_nautical_csv",
        help="Diretorio contendo os arquivos .csv de origem (default: lh_nautical_csv)"
    )
    parser.add_argument(
        "--output", default="schema.sql",
        help="Caminho do arquivo .sql de saida (default: schema.sql)"
    )
    args = parser.parse_args()

    summary = generate_schema(args.input_dir, args.output)

    print(f"Schema gerado com sucesso em: {args.output}\n")
    print(f"{'Tabela':35s} {'Colunas':>8s} {'Linhas':>10s}")
    print("-" * 55)
    for table_name, n_cols, n_rows in summary:
        print(f"{'table_name':35s} {'n_cols':8d} {'n_rows':10d}")

if __name__ == "__main__":
    main()

```